

HeteroPod: 面向通用云原生应用的 XPU 加速基础设施卸载

Bicheng Yang Jingkai He Dong Du* Yubin Xia Haibo Chen

上海交通大学 IPADS; 教育部领域专用操作系统工程研究中心; 中国科学院系统软件重点实验室

摘要

云原生系统日益依赖服务网格、监控智能体等基础设施服务; 这些服务会与用户应用争用资源, 进而损害性能与可扩展性。本文提出 HeteroPod: 一种把此类服务卸载到数据处理单元 (DPU) 的新抽象, 以实施严格隔离, 同时减少主机资源争用与运维成本。为实现 HeteroPod, 本文引入跨处理单元 (XPU) 网络系统 HeteroNet, 其包含两项关键能力: (1) 拆分网络命名空间, 为横跨 CPU 与 DPU 的进程提供统一网络抽象; (2) 弹性高效的 XPU 网络, 在不引入固定资源开销和轮询成本的前提下达到共享内存性能。借助 HeteroNet 以及云原生工作负载的组合式特征, HeteroPod 能够以合适方式把基础设施容器卸载到 DPU。我们基于 Linux 实现 HeteroNet, 并基于 Kubernetes 实现云原生系统 HeteroK8s。我们使用 NVIDIA BlueField-2 DPU 和基于 CXL 的 DPU (以真实 CXL 内存设备模拟) 评估这些系统。结果表明, HeteroK8s 能有效支持复杂且未经修改的通用云原生应用 (代码规模最高达 100 万行); 与内核旁路设计相比, 其延迟最高改善 31.9 $\times$ 、资源消耗最多降低 64 $\times$; 与现有先进系统相比, 其端到端延迟最多改善 60%, 可扩展性最高提升 55%。

1 引言

云原生计算已经成为云和数据中心环境中的重要范式, 它利用容器化来分发和部署应用 [3–5, 10, 19]。执行环境的基本单元, 例如 Kubernetes 中的 *Pod*¹, 通常包含一个或多个容器; 每个容器拥有独立镜像, 但共享网络环境。借助这种方式, 应用被封装成可移植、自包含的单元, 并可部署和分发到不同节点。

相较非云原生应用, 云原生应用的一项重要优势是其开发过程得到大量云基础设施服务的支持, 包括存储、网络与安全等。这些基础设施服务同样以容器部署。基础设施容器可分为两类: 全局基础设施容器和逐 Pod 基础设施容器, 如图 1(a) 所示。例如, 平台调度器是一种全局基础设施容器, 负责把请求编排到不同 Pod; 边车代理 (如 Envoy[22]) 则是一种存在于每个 Pod 中的逐 Pod 基础设施容器, 为应用提供网络功能。

基础设施容器虽然对发挥云原生计算优势至关重要, 却也带来不可忽视的资源负担, 本文称之为基础设施负担 (*infra-burden*)。例如, 服务网格 [1] 已经成为云原生平台管理微服务的一种常见模式, 它使用专用代理 (称为边车代理) 代替应用处理网络流量。由于每个微服务 Pod 都包含一个作为基础设施容器的边车代理, 这些代理容器可能消耗节点上最高 25% 的 CPU 和 33% 的内存资源 [66]。基础设施负担问题已持续很长时间 [54], 而且非常难以缓解 [43, 51, 82]。

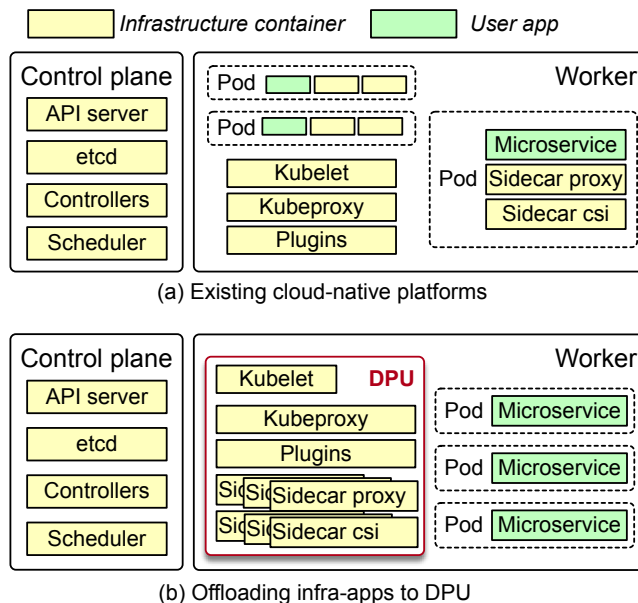


图 1: 包含基础设施容器和用户应用的云原生平台。

近期一个可能带来缓解的重要趋势是, 云厂商正在部署越来越多的异构设备, 以提高可扩展性、性能与成本效率 [2, 8, 9, 11, 12, 16, 45, 56, 81]。数据处理单元 (DPU) 是一类新型异构设备, 它通过嵌入式处理单元和高效 NIC 提供可扩展、节能的计算能力 [65]。DPU 有望显著提高单节点的实例密度, 例如在无服务器计算中提升 50% [44]。隔离的内存和核心还为这些设备提供了更强的安全性, 而大量位于数据路径上的硬件线程可提升分组处理能力; 例如 BlueField-3 支持 256 个硬件线程 [73]。预期 DPU 等异构处理单元将通过 PCIe 6.0、CXL [6] 等高速互连连接, 从而进一步提升性能。利用 DPU 因而可能缓解基础设施负担。

目标: 把云原生基础设施服务卸载到 DPU。 传统方法会在 CPU 和 DPU 上共同放置基础设施容器与用户应用 [18]; 与之不同, 我们提出一种很有潜力但也颇具挑战的体系结构: 把这两类云原生容器解耦到不同处理单元 (PU), 如图 1(b) 所示。该策略通过避免基础设施容器和应用容器相互干扰, 可以有效缓解基础设施负担。然而, 把基础设施容器与应用容器分开也带来显著挑战。

挑战 1: 同一 Pod 中的基础设施容器与应用容器耦合在同一个网络命名空间中, 并依赖单 PU 操作系统。 共享 Pod 内的容器使用底层操作系统内核提供的网络命名空间。因此, 不同容器中的进程共享网络设备, 并可使用 “localhost” 进行跨容器通信。把基础设施容器卸载到 DPU 会破坏这种共享网络命名空间能力。在把基础设施容器和应用容器分布到不同 PU 的同时保持本地网络抽象, 是一个突出难题。

挑战 2: 把容器拆分到不同 PU 会在通信性能与资源利用

*通讯作者: Dong Du (Dd_nirvana@sjtu.edu.cn)。

¹ 本文把 Pod 视为云原生平台基本计算单元的广义表示。

率之间形成权衡。基础设施容器与应用容器将通过网络通信。由于网络分组需要经过两个 NIC, 相较采用 Linux 网络栈的单 PU 设计, 这种拆分可能使通信延迟增加 $10\times$ 到 $100\times$ 。rsocket[28] 等内核旁路机制 [23, 28, 31, 52, 59] 可以提供兼容的套接字级 API, 并直接利用 RDMA 等硬件能力获得理想性能。然而, 这些方案往往面临资源利用不佳的问题 [89], 包括为用户态链路分配固定内存造成的浪费, 以及用户态轮询浪费的 CPU 周期。SMC-R[46] 等内核辅助方法则受上下文切换和数据复制开销影响。因此, 系统迫切需要一种跨 PU、基于网络的通信方式, 在提供高性能的同时最大化资源利用率; 当前尚无这样的方案。

挑战 3: 并非所有基础设施容器都适合卸载到 DPU。 这一挑战源于 DPU 的固有特征: DPU 通常只包含面向特定领域任务 (如网络分组处理) 优化的低性能 SoC。既有研究表明, 直接把容器卸载到 DPU 的简单设计可能带来显著性能开销 [44, 54, 84]。因此, 系统设计者仍缺少有效卸载云原生基础设施的实践方法。

本文提出跨 PU Pod 设计 HeteroPod, 并以 HeteroNet 解决前两个挑战, 实现透明、高性能且资源高效的 XPU 网络。HeteroNet 引入两项关键技术: 拆分网络命名空间 (hetero-netns) 和弹性高效的 XPU 网络 (hetero-socket)。hetero-netns 为跨 PU 容器提供统一网络抽象; 其新意在于通过在多个 PU 上重新执行网络规则来建立拆分网络命名空间, 使直接和间接通信都能照常进行。其次, hetero-socket 是一种同时达到内核旁路性能、高资源效率和套接字兼容性的网络栈。hetero-socket 采用推测式分配来缓解内存浪费: 其两阶段按需缓冲区分配允许进程以绕过内核的方式从共享池中推测式申请新缓冲区, 之后再由内核提交该分配。此外, hetero-socket 支持用户态 NAPI 和跨 PU 操作系统协作, 利用页表与异常为用户态库提供中断能力, 从而降低轮询成本。

我们利用 HeteroPod 实现了基于 Kubernetes 的云原生系统 HeteroK8s; 该系统支持无服务器计算 (HeteroK8s-FaaS) 和带服务网格的微服务 (HeteroK8s-Mesh [24])。HeteroK8s 可以有效卸载边车代理等逐 Pod 基础设施容器和调度器等全局基础设施容器, 同时也足够通用, 能够卸载无服务器场景中合适的用户应用以提高系统密度。因此, HeteroK8s 也为第三个挑战提供了卸载基础设施容器的实践准则。

我们在配有 NVIDIA BlueField-2 DPU 的真实平台和基于真实 CXL 内存设备构建的 CXL-DPU 模拟平台上评估全部原型。结果表明, hetero-socket 的延迟最高改善 $31.9\times$, 资源消耗最多降低 $64\times$ 。与商用 Istio 相比, HeteroK8s-Mesh 的可扩展性最高提升 55% , 端到端延迟最多改善 60% ; 与基线系统相比, HeteroK8s-FaaS 的通信延迟最高降低 $20\times$, 应用延迟最高降低 $4.45\times$ 。HeteroK8s 还能把控制面调度器卸载到 DPU, 在那里缓存全局镜像状态并实现镜像感知调度, 使平均延迟降低 $14.4\times$ 。

2 动机

2.1 云原生应用及其问题

云原生应用通常部署在 Pod 中; Pod 是 Kubernetes 等平台采用的多容器抽象。如图 1(a) 所示, 我们把云原生容器分为基础设施容器和用户容器两类。相较非云原生应用, 云原生应用的一项潜在优势在于, 其开发得到庞大的

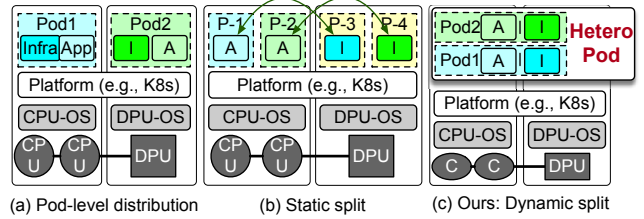


图 2: CPU-DPU 计算机上的云原生应用。

云基础设施服务支持, 包括存储、网络和安全等。例如, 云厂商通常为每个微服务 (用户容器) 部署 Envoy[22] 作为基础设施容器, 以管理网络。我们强调当今这种云原生体系结构中的两个问题 (I)。

I1: 基础设施容器给 CPU 和内存带来显著负担。 基础设施容器首先会引入不可忽视的资源负担。例如, 服务网格 [1] 使用专用的边车代理代表应用处理网络流量, 是管理微服务的一种常用模式 (图 1(a))。我们用 emoji-voting[21] 微服务评估 Kubernetes 上边车容器引入的资源成本 (详细设置见第 5.1 节)。结果显示, 代理容器可能消耗节点上最高 25% 的 CPU 和 33% 的内存资源。此外, 即使 RPS 很低 ($RPS=20$), 代理容器仍可能消耗大量资源, 进一步加剧节点中的资源干扰。

I2: 对模块化云原生应用而言, 通信延迟更加重要。 微服务和无服务器函数都是细粒度、模块化应用: 一个应用可能被划分为几十乃至数千个微服务或函数, 它们依赖通信机制进行协作。由于每项服务的执行时间只有几毫秒, 通信延迟对端到端性能十分重要。事实上, 边车代理等基础设施容器甚至会进一步增加延迟: 使用代理 Envoy 后, 延迟可能增加 $9.5\times$ ($RPS=20$) 到 $49.8\times$ ($RPS=3K$) [66]。为了规避这项成本, 目前常见的做法是把一条函数链 (或高度相关的微服务) 部署在同一节点, 并依靠 IPC 或域套接字等本地通信方法降低延迟 [34, 44, 53, 58]。这导出一项重要要求: 提高用户应用密度, 而且越高越好。但基础设施容器的干扰使这一目标很难实现。

2.2 面向云原生应用的 CPU-DPU 平台

DPU (数据处理单元) [14, 68, 70] 是一类在数据中心内移动和处理数据的新型可编程设备。DPU 是能够运行商用操作系统以及网络代理等性能关键程序的 SoC, 通常包含多核处理单元 (例如 BlueField-2 DPU 中 2.75GHz 的 ARM 核心) 和高效网络接口。既有工作已经证明, DPU 可以提高无服务器函数密度 [44]、降低分布式文件系统的延迟和资源成本 [54], 并实现节能微服务 [65]。

将 DPU 用于云原生应用的既有尝试。 如图 2(a)(b) 所示, 商用云平台通常采用 Pod 级分布, 例如 NVIDIA DPU Container[27] 和 Red Hat OpenShift[18]; 或者采用静态拆分 [54, 62, 78], 例如 Azure[45]。Pod 级分布把 DPU 当作独立服务器, 直接在 DPU 上部署未经修改的应用。然而, 应用需求与硬件能力可能不匹配, 例如 DPU 往往只有低性能核心, 因而应用可能出现显著性能下降。静态拆分则把应用拆成细粒度组件并分发到不同 PU, 通过通信协作; 它只把适合的组件部署到 DPU, 例如让 CPU 运行计算密集组件、DPU 运行网络密集组件, 因而能取得良好性能。但这种方法通常要求在开发阶段理解应用和硬件, 难以适配通用应用与多样的异构设备。尽管如此, 两类方法在真实工业应用中都具有实用性, 并能提供特定收益。

近期研究针对无服务器计算 [44]、微服务 [65] 等特定

云原生场景开发了新的异构框架。然而，这些框架通常要求修改应用，因此只能支持简单应用（例如 Molecule[44] 只支持少于 1.8K 行 Node.js 代码），对于 Envoy 这类复杂通用应用则不够实用；Envoy 是广泛使用的服务网格代理，包含 764K 行 C++ 代码。许多通用云原生应用以二进制或容器镜像形式分发，很难修改，因此上述限制尤为显著。

本文方法：使用 HeteroPod 动态拆分。 与既有方法不同，我们提出动态跨 PU 解耦：在运行时透明地把容器（即使属于同一 Pod）划分到不同异构 PU，如图 2(c) 所示。我们把这种体系结构称为 HeteroPod（跨 PU Pod），它解决了传统部署中的关键局限。第一，把基础设施容器隔离到 DPU 可以消除其与应用容器的资源争用。第二，把合适的用户容器也卸载到 DPU，可帮助提高单节点密度。第三，也是最重要的一点，与静态拆分不同，HeteroPod 的动态拆分保留现有 Pod 语义，因此能支持未经修改的通用云原生应用。

3 HeteroPod 设计概览

HeteroPod 的核心思想很简单：选择性地把基础设施容器和应用容器部署到 DPU，其余容器继续留在 CPU 上。这种容器粒度部署利用每个容器的自包含特征简化解耦。不过，关键挑战在于跨容器通信，因为云原生平台中的容器依赖网络交互。

具体而言，HeteroPod 必须解决两项技术挑战：（1）提供透明的跨 PU 网络，从而支持未经修改的应用，即第 1 节的挑战 1；（2）实现高效率的跨 PU 通信，即第 1 节的挑战 2。为此，我们把 HeteroNet 设计为 HeteroPod 的基础技术；HeteroNet 为跨 PU（XPU）网络引入两项关键创新。

第一，HeteroNet 实现 hetero-netns，为跨 PU 容器提供统一网络抽象。hetero-netns 以全局端口空间为指示器来支持跨 PU 直接连接，并利用如下观察支持跨 PU 间接通信：只要不引入新的副作用，网络过滤器或规则就可冗余地应用于源 PU 和目标 PU。由此，hetero-netns 创建一个拆分网络命名空间，在不同 PU 的组件之间保留通信语义。第二，HeteroNet 引入 hetero-socket，这是一种内核与库协同设计的网络栈，同时实现内核旁路性能、内核辅助的资源效率和套接字 API 兼容性。hetero-socket 采用两项关键机制：推测式分配是一种两阶段缓冲区管理策略，进程先从共享池临时领取缓冲区（绕过内核），再由内核验证并提交分配；用户态 NAPI 利用跨 PU 操作系统协作，使用户态库可以通过中断驱动处理事件，从而显著减少轮询开销。

4 HeteroNet 的设计与实现

4.1 拆分网络命名空间（hetero-netns）

记号。 Linux 的网络命名空间 [25] 会隔离网络设备、协议栈、IP 路由表和端口号等网络相关系统资源。我们把这一概念扩展为拆分网络命名空间，在 CPU-DPU 计算机内提供统一且相互隔离的网络抽象。一个拆分网络命名空间可以由一个容器（多个进程）独占，也可以由多个容器共享，例如 Kubernetes Pod。

网络调用。 网络需求可分为两类（图 3(a)(b)(c)）：直接调用和间接调用。直接调用指网络命名空间内的两个进

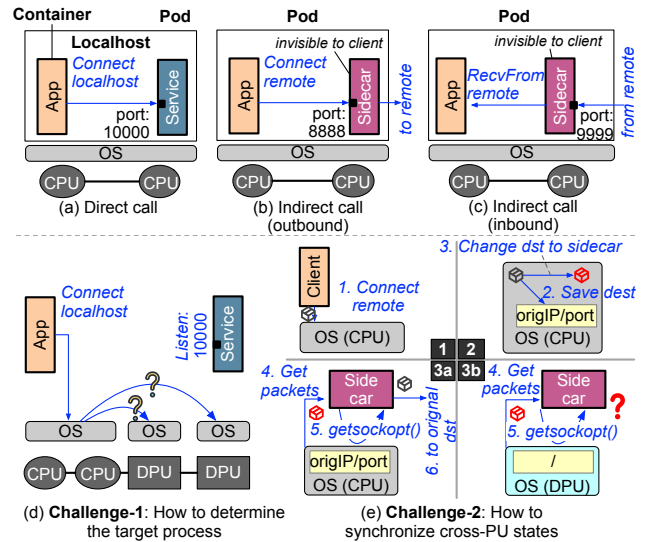


图 3：两类网络调用以及拆分网络命名空间的挑战。

程使用 localhost 和端口号进行通信。间接调用则指消息并非直接发给本地接收方，而是最初准备发送给远程接收方，随后因 iptables 或路由等分组过滤规则被转发到本地接收方。

挑战。 当多个内核管理“同一个”网络命名空间中的进程时，首先会出现新的困难。对于直接调用，发送方操作系统内核难以确定哪个内核负责接收方进程。一种朴素方案是让发送方内核把分组转发到接收方内核，但如图 3(d) 所示，发送方并不知道各进程部署在哪个 PU。

其次，间接调用又引入状态同步问题：不同 PU 上的操作系统内核可能必须显式同步某些状态，才能正确处理分组。以图 3(e) 为例，系统部署边车代理容器来截获应用容器的入站与出站分组。通常，系统会向 Pod 的网络命名空间加入 iptables 规则，把应用发送的分组重定向到代理监听的端口。这些规则把目标 IP 和端口改成代理的地址，并由操作系统内核保存原始目标 IP/端口，供以后使用（图 3(e) 的步骤 2/3）。代理容器收到分组后进行过滤，并调用 getsockopt() 系统调用读取原始目标 IP/端口（步骤 4/5）。一种朴素方案是在发送方（即应用容器）加入类似规则，把目标 IP 和端口改成预期接收方所在 PU 的地址。然而，位于另一 PU 的代理容器无法取得原始 IP 和端口，因为关键数据保存在发送方 CPU 的内核中，而非接收方 DPU 的内核中。理论上可以在两个 PU 之间同步这些数据，但规则与状态十分复杂，实践中很难完成。

4.1.1 基于端口的反向映射

我们观察到，在拆分网络命名空间中，分布在不同 PU 上的进程仍然共享同一个逻辑端口空间，每个端口号只能由一个进程独占。基于这一观察，我们提出基于端口的反向映射。该方法维护一张记录端口号及其对应 PU 的反向映射表，且这张表由每个 PU 上的全部 HeteroNet 守护进程全局共享。当容器需要使用某个端口号时，HeteroNet 守护进程检查表项以确保端口可用；如果端口已被占用，操作就会失败。

为避免在数据面同步和检查端口的开销，HeteroNet 解耦控制面与数据面。端口信息发生变化时，HeteroNet 守护进程生成并应用一组单 PU 分组过滤规则；原型主要使用 iptables 和路由。这些规则自动检测远程端口号，

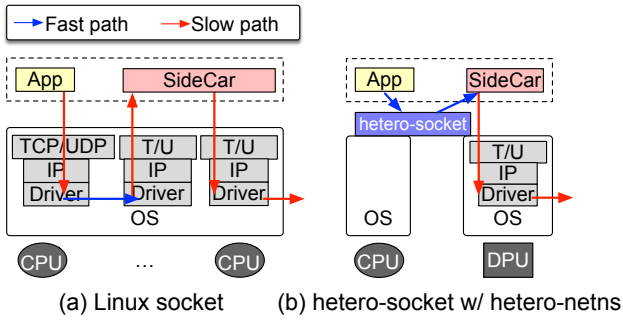


图 4: 包含基础设施容器时的通信路径。

并利用 DNAT（目标网络地址转换）和 SNAT（源网络地址转换）把分组正确转发到相应 PU。因此，发送和接收进程都感知不到底层变化，仍可像未发生拆分一样使用 localhost 通信，并且成本几乎为零。

4.1.2 部分幂等的分组规则

为解决第二项挑战，我们提出部分幂等的分组规则。系统不再只在源 PU 执行分组规则、随后再同步状态（这很困难），而是把原始规则转换为两组分别应用于源 PU 和目标 PU 的新规则。这些规则可能对分组执行冗余操作，但不会引入新的副作用，因而在此意义上是幂等的。通过在两个 PU 上重新执行操作，状态可自动实现同步。

以图 3(e) 为例，系统不再只修改目标地址为接收方 IP/端口并在发送方 PU 保存原始目标地址，而是将过程简化为三条规则：（1）在发送方 PU 上，把分组路由到接收方 PU，但不修改分组；（2）在接收方 PU 上，把分组路由到接收容器；（3）在接收方 PU 上修改目标地址并保存原始目标地址。规则（1/2）和规则（3）都会把分组移向接收方 PU，因而存在冗余操作，但它们不会改变行为，并且实现了状态同步。间接调用通过网络规则实现，因此 hetero-netns 能检测这些规则，并用新规则替换入站和出站流量的截获规则。

4.2 弹性高效的 XPU 网络

4.2.1 概览

传统网络栈给 XPU 通信引入不必要的重复路径，导致延迟增加，并限制高速互连的优化空间。例如，在包含边车的微服务中，一条消息必须三次经过网络栈才能到达目的地，如图 4 所示。虽然已有兼容套接字的内核旁路系统 [7, 23, 31]，但固定内存缓冲区和静态分配使其资源利用率很低（第 4.2.2 节）。hetero-socket 的目标是在 XPU 场景中克服这些资源问题，同时达到既有内核旁路设计的性能。

把跨 PU 互连抽象为全局共享内存。 我们将 XPU 通信所需的一项关键原语概括为全局共享内存 (GShm)，即使用一块共享内存作为跨 PU 通信通道。现有 DPU 已满足该假设 [15, 68]；例如，BlueField-2 支持 RDMA，使 CPU 和 DPU 可以共享显式分配的内存区域。为适配不同互连，hetero-socket 引入互连抽象层 (IAL)，它隐藏互连细节，只向 hetero-socket 核心暴露 GShm 抽象。

控制路径与数据路径解耦。 hetero-socket 依靠 Linux 网络栈和 hetero-netns 建立连接；如果 GShm 可用，系统随后创建基于 GShm 的通道以取得最佳性能。运行时，`read()` 和 `write()` 等数据路径 API 会在可行时被拦

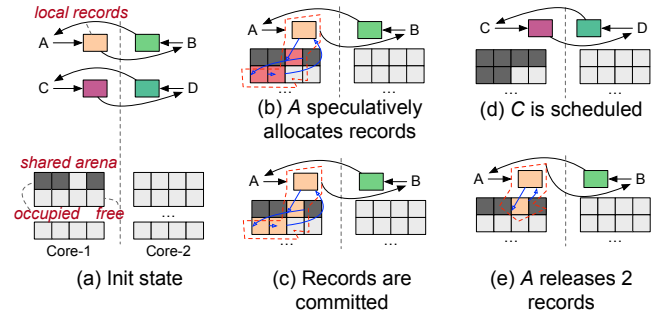


图 5: 推测式分配流程：(a) 记录与共享区域；(b) 推测式分配；(c) 内核提交；(d) 访问控制；(e) 释放。

截并改用 GShm，但应用仍可调用 `getsockopt()` 等控制路径操作来取得连接元数据。当容器部署在不同 PU 上时，hetero-socket 先利用套接字与 hetero-netns 正确建立连接，再使用 GShm 加速通信。

4.2.2 挑战

内核旁路和基于共享内存的设计较为常见，也能充分利用硬件能力而获得最佳性能；hetero-socket 进一步处理两项尚未解决的技术挑战。第一项是**固定资源成本**。分配共享内存区域通常是特权操作，需要两个 PU 的操作系统用互连原语进行协商。该过程通常很慢，例如 RDMA[85]，因此难以动态扩展区域。现有系统往往为每条连接在操作系统中预留固定大小的区域，在资源浪费与性能下降之间权衡。真实机器中可能存在数千条连接，因此问题尤其严重。第二项是**轮询成本**。由于操作系统内核被绕过，用户态网络栈必须显式轮询数据，再次落入 PU 开销与长延迟之间的传统权衡。

4.2.3 采用推测式分配的用户态资源管理

hetero-socket 通过操作系统与库协同设计来高效管理内存（图 5）。其核心抽象是记录 (record)：一种页对齐、带有 `nextRecord` 等元数据的缓冲区，由发送或接收进程单向访问。每条连接维护两个本地记录（每个方向一个），同时使用逐核共享区域 (arena)；共享区域包含可以动态分配的记录，并由 `headRecord` 与 `tailRecord` 组成的列表管理。这种混合方法以本地记录保证性能、以共享区域保证可扩展性，并解决以下三项关键问题。

隔离与同步。 共享区域需要严格访问控制：进程只能访问属于自己连接的记录，或其所在核心的空闲记录。hetero-socket 通过内存映射实施这一约束。调度进程时，操作系统映射（1）本地记录；（2）活动连接占用的共享记录；（3）当前核心的空闲记录。由于进程时间片内只有它能访问所在核心的空闲记录，逐核共享区域消除了同步开销。

动态记录串接。 记录通过 `nextRecord` 元数据构成链表。发送方用尽缓冲区空间时，无需内核参与，即可推测式地把当前核心共享区域中的空闲记录追加到链表（图 5(b)）。内核随后在上下文切换或异常（如缺页）期间提交这些分配，通过重新映射把记录标记为已占用（图 5(c)(d)）。这种“用户空间推测、内核随后认可”的两阶段方法达到内核旁路性能，同时把固定内存减少 $64\times$ （第 6.1 节）。连接的接收进程访问发送方刚分配的记录时，因为没有访问空闲记录的权限，会触发缺页异常。此时，操作系统内核通过把所有新分配记录映射给接收方来处理缺页，同

时完成提交。实践中这种陷入很少发生。

资源治理。 限速策略对每个进程可使用的记录数设置上界，以防止资源囤积。内核在提交阶段实施这些策略，同时保留逐核分配的优势。

设计总结。 因此，除把全局资源池改成逐核池（共享区域）外，hetero-socket 能达到以内核为中心的设计所具有的资源管理效果。该改变可避免跨核心干扰和同步，因而是合理的。与预留内存的内核旁路网络设计相比，接收方陷入极少发生，所以 hetero-socket 可取得相近性能。我们甚至观察到性能有所提升，因为 hetero-socket 的动态资源能更好地在连接之间调度，并适应变化的负载。

4.2.4 用户态 NAPI

hetero-socket 引入 U-NAPI，即 Linux NAPI[32] 的用户空间版本；它通过中断驱动的事件处理消除轮询开销。传统内核旁路设计不断轮询事件，浪费 CPU 周期；U-NAPI 则仿照操作系统内核中 NAPI 的自适应行为，根据流量模式在中断与轮询模式之间动态切换。

事件管理。 hetero-socket 保持标准 epoll 语义，并增加两个关键系统调用：enable_notify_gshm 和 disable_notify_gshm。应用通过它们向内核注册所关心的事件，例如缓冲区可用。enable_notify_gshm 通过不同标志支持 sync 和 async 两种模式。sync 会阻塞线程直到事件发生，适合阻塞 I/O；async 会触发信号处理程序，由后者关闭中断并在延迟敏感阶段启用轮询。这里的关键创新是，U-NAPI 利用操作系统内核模拟硬件，为用户态事件管理提供中断能力。

与网络操作集成。 实践中，U-NAPI 优化三类常见情形。第一，网络 write（或 send）操作可能因为连接缓冲区已满而阻塞并转为轮询。此时，hetero-socket 调用 enable_notify_gshm(token, sync | W_flag)，在内核中阻塞，直到连接可以写入。第二，空连接上的阻塞式 read（或 recv）情形类似，hetero-socket 可使用带 R_flag 的 sync 模式。第三，当需要向 epoll 实例加入新事件时，hetero-socket 可用 async 模式监控事件，即调用 enable_notify_gshm(token, async | events)；随后，信号处理程序可处理事件更新、关闭通知，并为性能转回轮询。

跨 PU 事件传播。 U-NAPI 使用两种跨 PU 事件传递技术。第一种是主动通知：连接远端一侧主动告知远端（或同一 PU 的）内核出现新事件。例如，在阻塞写入情形中，远端接收方读取数据、检查状态并发现发送方被阻塞后，会用系统调用通知内核连接状态已变化，最终提醒发送方可能已有可用空间。第二种是读写时通知（Notify-on-RW, NoRW）：内核通知远端（或同一 PU 的）内核撤销连接记录权限，例如在写入情形中撤销读权限。这样，接收方读取时会触发缺页异常，异常再被送到发送方内核，以通知连接状态变化。跨 PU 延迟虽然高于本地情形，但应用仍可在关键路径上灵活退回轮询。

4.3 实现

hetero-netns。 hetero-netns 以守护进程形式运行在每个采用 Linux 的 PU 上，监控本地网络规则，并与同一计算机中的其他 PU 协作。它维护端口信息，并

截获网络命名空间中的网络规则。hetero-netns 复用现有网络技术，例如 nsenter、NAT、策略路由和 iptables。在 HeteroK8s 中，我们用 hetero-netns 扩展 Kubernetes CNI。

hetero-socket。 hetero-socket 使用 C/C++ 编写。我们借助 LD_PRELOAD[13] 在运行时替换应用中由 libc 实现的套接字相关函数。此外，我们在 CXL-DPU 上以 CXL、在 BlueField-2 上以 RDMA 实现全局共享内存（GShm）。hetero-socket 实现了一整套 Linux 套接字接口；我们还扩展 Linux 内核，使其支持推测式分配和 U-NAPI 接口。

多指令集体系结构支持。 我们使用多平台容器镜像功能 [26] 支持同一计算机中的多 ISA PU；这是 Docker/Kubernetes 的标准功能。

安全性。 当前 HeteroNet 以逐进程库实现，在用户空间维护两个实体之间的全局共享内存。每个进程只能访问属于自身连接的共享内存，因此 HeteroNet 不会引入攻击其他进程的新安全漏洞。恶意进程可能破坏自己的连接，但由于指针操作经过谨慎处理，无法造成溢出等其他问题。此外，管理员可通过配置文件指定哪些连接允许使用 hetero-socket，其作用类似固件中的显式允许列表。

实验设置。 评估使用两种设置。第一种是 **BlueField-DPU**：我们使用两台服务器，每台配有两颗 E5-2650 v5 CPU（每颗 12 核、2.2GHz）、96GB DDR4-1600 内存和一块 NVIDIA BlueField-2 DPU。每块 BlueField-2 DPU 包含 8 个 ARM Cortex-A72 核心（2.75GHz）、16GB DDR-1600 DRAM 和带两个 100Gbps RDMA 端口的 ConnectX-6 NIC；我们也在 BlueField-3 上验证结果。第二种是 **CXL-DPU**：一台双路服务器配有 Intel Xeon Platinum 8468V（2.9GHz、48 核），并通过 CXL 2.x 内存扩展卡连接一条 32GB、4800MT/s 的 DDR5 RDIMM。我们用两个虚拟机分别表示 CPU 和 DPU，并把真实 CXL 设备配置为 dax 设备，作为 GShm，从而实现基于 CXL 的 DPU。

5 HeteroK8s：XPU 云原生系统

我们提出 HeteroK8s：一个基于 Kubernetes 的云原生系统，它利用 HeteroNet 在不同场景下有效且高效地使用 DPU。下面给出三个案例研究：卸载工作节点的基础设施应用（第 5.1 节）、卸载用户应用（第 5.2 节），以及卸载控制面的基础设施应用（第 5.3 节）。

5.1 案例研究：服务网络与微服务

服务网络是一种代表性模式，它会向每个微服务 Pod 注入一个边车容器（代理），从而同时带来资源成本和通信延迟问题。下面说明 HeteroK8s 如何利用 HeteroNet 优化代理类基础设施容器造成的成本。

采用 Istio 的基线系统。 除基本代理设计（图 6(a)）外，先进服务网络系统 Istio[24] 还支持一种能有效缓解成本的高级体系结构：waypoint 边车 [17]（图 6(b)）。它把边车解耦成零信任隧道（ztunnel）与 waypoint 边车。ztunnel 可由节点上的全部应用共享，用于执行 L4 策略；waypoint 边车则专用于某个应用、执行 L7 策略，并可被调度到其他节点。解耦能节省应用节点上的资源，但把 waypoint 边车调度到第三台服务器也会增加正常路径的延迟。

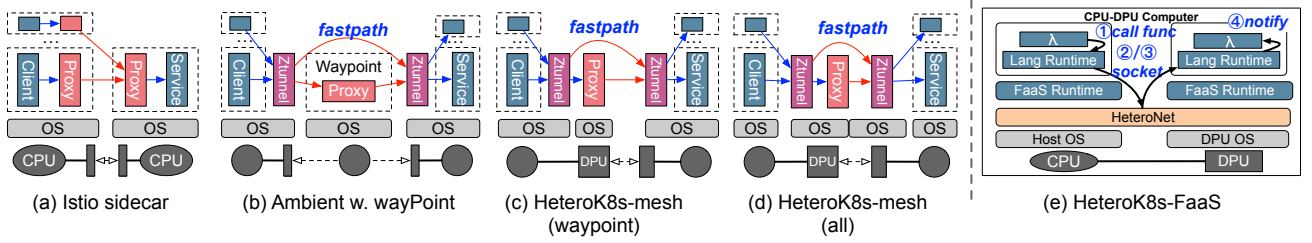


图 6: 采用不同体系结构的服务网络 (a/b)、HeteroK8s-Mesh (c/d) 和 HeteroK8s-FaaS (e)。

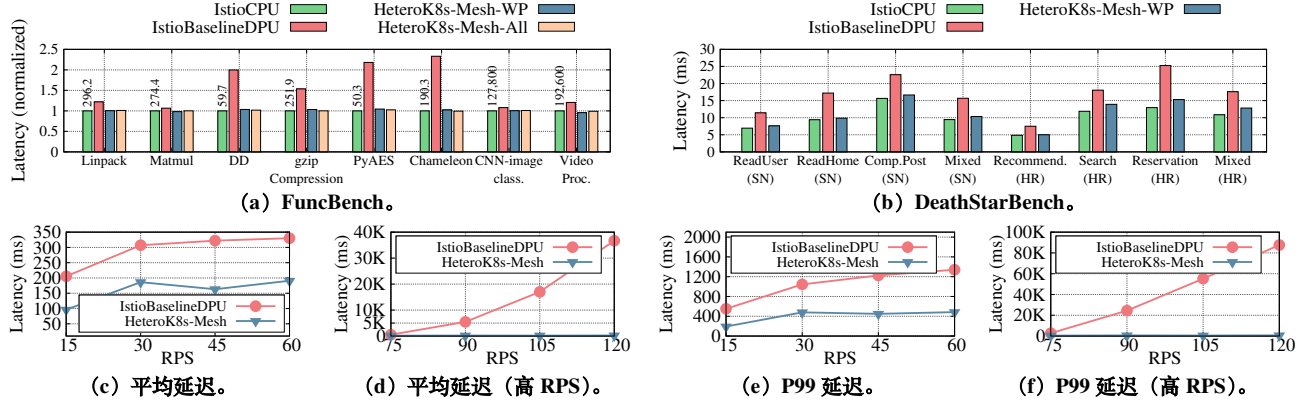


图 7: BlueField-2 DPU 上带服务网格的微服务。DeathStar 中 SN 表示 Social Network, HR 表示 Hotel Reservation。图中给出端到端延迟; 除非明确说明, 单位均为毫秒。

HeteroK8s-Mesh。我们设计 HeteroK8s-Mesh, 使用 HeteroNet 把边车高效卸载到 DPU, 并在性能和可扩展性之间取得平衡。HeteroK8s-Mesh 支持两种部署方式: (1) 把全部代理相关基础设施容器 (ztunnel 和 waypoint) 卸载到 DPU, 使 CPU 专注运行微服务应用 (图 6(d)); (2) 只把 waypoint 容器卸载到 DPU (图 6(c))。当 DPU 设备故障或资源不足时, 系统会在 CPU 上启动新的 ztunnel 和 waypoint 容器, 以保证微服务可用性和服务质量。如果客户端与服务端微服务位于同一节点, HeteroK8s-Mesh 会尽可能把 ztunnel 和 waypoint 容器部署在 CPU 上。HeteroK8s-Mesh 使用 hetero-netns 支持应用与代理之间的间接通信, 并依靠 hetero-socket 降低通信延迟。

方法。我们以 DeathStarBench[48] 和 FunctionBench[55] 作为微服务部署, 并为每个微服务部署一个采用 Envoy 的 Istio waypoint 作为网关, 以评估 HeteroK8s-Mesh 的性能。微服务的全部入站与出站流量都由节点上的 ztunnel 截获和处理。我们将 HeteroK8s-Mesh 与两个系统比较: IstioCPU 和 IstioBaselineDPU。IstioCPU 只在 CPU 上部署微服务 Pod, 在许多情形中代表最佳性能, 但不能有效利用 DPU 提高可扩展性。IstioBaselineDPU 基于 Istio 实现, 同时在 CPU 和 DPU 上部署微服务 Pod, 并优先调度到 DPU; 它代表以 Pod 粒度使用 DPU 的既有方法。

端到端性能。我们在 HeteroK8s-WP (图 6(c)) 和 HeteroK8s-All (图 6(d)) 两种配置下评估微服务的端到端延迟。为单独展示动态拆分的收益, 此处 HeteroK8s-Mesh 不使用 hetero-socket。

FunctionBench 的评估结果见图 7(a)。与 IstioBaselineDPU 相比, HeteroK8s-Mesh 可使微服务端到端延迟最多降低 60% (dd, pyaes, chameleon), 并改善全部应用的性能。与仅使用 CPU 相比, 只把 waypoint 卸载到 DPU 的 HeteroK8s-Mesh-WP 带来的端到端延迟开销低于 5%。把 ztunnel 和 waypoint 都卸载到 DPU 的 HeteroK8s-Mesh-All 可以节省更多 CPU 资源, 其端到端开销在 dd 和 pyaes

上仅为 1%~2%, 在其余应用上均低于 1%。HeteroK8s-Mesh-All 的端到端延迟开销之所以低于 HeteroK8s-Mesh-WP, 是因为服务网格基础设施完全卸载到 DPU 后, CPU 上运行的微服务面临更少的 CPU 和缓存、TLB 等微体系结构资源争用, 因而运行时间缩短。

DeathStar 的评估结果见图 7(b)。由于 DeathStar 微服务自身的运行延迟很短, 此处不卸载 ztunnel。与 BaselineDPU 相比, HeteroK8s-Mesh 的延迟改善 29%~74%; 与 CPU (不卸载时的理想性能) 相比, 它平均只增加不到 1.2ms 的延迟。

可扩展性。我们比较高密度部署下的可扩展性。具体而言, HeteroK8s-Mesh 与 IstioBaselineDPU 都使用全部 DPU 资源和大部分 CPU 资源部署 Pod。每个微服务 Pod 都配有一个 waypoint 代理; 微服务和 waypoint 实例总数相同, 均为 70, 分配的资源也相同: 每个 waypoint 使用 128MB 内存和 100mCPU, 每个应用使用 256MB 内存和 300mCPU。测试用例为 Pyaes, 请求通过 Nginx 以随机方式均匀负载均衡到全部微服务。

平均延迟结果见图 7(c)(d), P99 延迟见图 7(e)(f)。在不同 RPS 下, 使用 HeteroK8s-Mesh 部署的微服务均优于 IstioBaselineDPU, 端到端延迟改善 39%。随着 RPS 上升, 基线方案中微服务延迟迅速增长, 而 HeteroK8s-Mesh 方案的延迟保持相对稳定。这说明 HeteroK8s-Mesh 的可扩展性优于基线。

性能分解。我们针对大型与小型应用进行性能分解, 把端到端延迟拆成五部分: 节点内通信 (包括 PU 内与跨 PU) 延迟、跨节点通信延迟, 以及 ztunnel、waypoint 和应用的延迟。如图 8 所示, 对于大型应用, 只采用 hetero-netns 的 HeteroK8s-Mesh 已取得良好性能: 比 IstioDPU 快 2×, 并且几乎与 IstioCPU 相同。此处处理成本占据延迟主导地位, 因此 hetero-socket 带来的改善很小。对于小型应用, 采用 hetero-netns 的 HeteroK8s-Mesh 优于 IstioDPU, 但 waypoint 延迟上升使

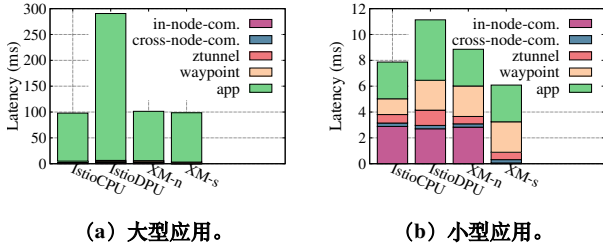


图 8: BlueField-2 DPU 上的性能分解。XM-n 表示使用 hetero-netns 的 HeteroK8s-Mesh, XM-s 表示使用 hetero-socket 的 HeteroK8s-Mesh。

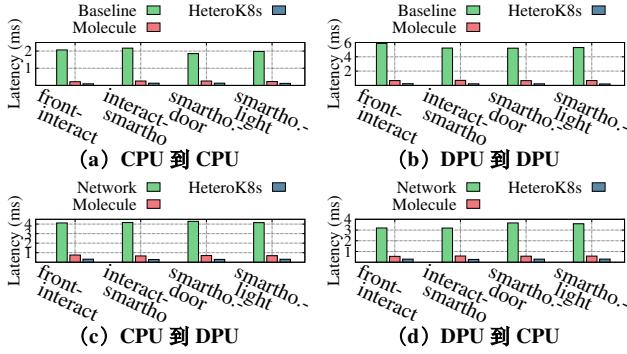


图 9: 无服务器 DAG 通信延迟。相较先进系统 Molecule, HeteroK8s-FaaS 的延迟降低 $2\times$ 。

其比 IstioCPU 慢 16%。同时采用 hetero-socket 的 HeteroK8s-Mesh 可通过优化通信进一步降低延迟, 性能甚至优于 IstioCPU。

5.2 案例研究: 无服务器计算 (FaaS)

近期工作 [42, 44, 77, 86] 已经尝试把 DPU 等异构设备用于无服务器计算, 以获得更好的可扩展性和性能, 例如 Molecule[44]。然而, 这些方法通常要求对无服务器运行时和应用都进行不可忽略的修改。

HeteroK8s-FaaS。 HeteroK8s 在 CPU-DPU 计算机上支持无服务器计算, 而且无需修改函数二进制、语言运行时或无服务器运行时 (图 6(e))。Molecule 使用同一无服务器运行时同时管理 CPU 和 DPU; 与之不同, HeteroK8s-FaaS 把不同 PU 视为独立设备, 并加入 HeteroNet 以透明地实现函数间高效通信。由于 Molecule 的函数采用单容器沙箱, HeteroK8s-FaaS 不使用 hetero-netns。

我们基于 Molecule 开源的同构实现构建 HeteroK8s-FaaS, 并使用其经过修改、支持容器 fork 以降低启动延迟的 runc。HeteroK8s-FaaS 为每个函数实例预加载 HeteroNet, 但通过配置文件只在同一函数链的实例之间启用 hetero-socket; 单函数应用不会使用 HeteroNet。HeteroK8s-FaaS 假设全局网关把一条函数链调度到同一台异构计算机。我们把 HeteroK8s-FaaS 与 Molecule 以及 Molecule-homo (CPU-only 计算机上的同构版本) 比较。

微基准。 我们通过微基准分析一条函数链中两个函数之间的通信延迟。以 Alexa Skills[88] 为例, 将 HeteroK8s-FaaS 与使用 nIPC 的 Molecule 以及使用 Node.js Express 的基线 (Molecule 的同构版本) 比较。评估考虑四种情形: 仅 CPU、仅 DPU、CPU 到 DPU 和 DPU 到 CPU, 结果见图 9。得益于高效的 nIPC 通信, Molecule 相对基线已经把延迟降低约 $10\times$ 。HeteroK8s-FaaS 的性能又高约 $2\times$, 因为 Molecule 需要额外执行一次 IPC 调用以进入 Hetero-

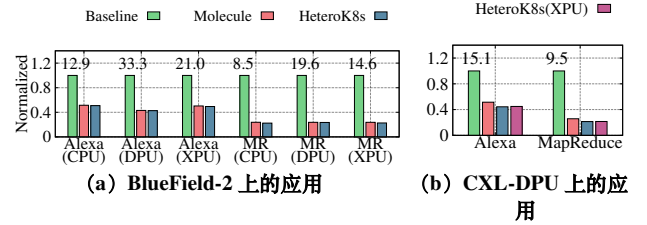


图 10: 链式应用。MR 表示 MapReduce, XPU 表示跨 PU。

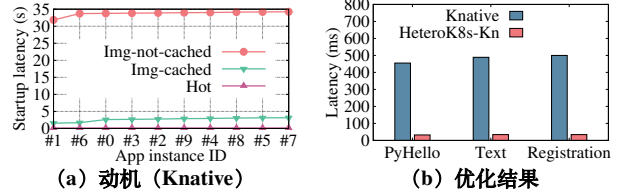


图 11: 卸载调度器以实现镜像感知调度。

Shim。由于本文实验环境的硬件配置更好, Molecule 的结果优于原文报告数据 [44]。

应用。 我们分析 ServerlessBench 与 FunctionBench 中两个链式应用 Alexa 和 MapReduce 的端到端延迟。Alexa 包含五个 Node.js 函数, MapReduce 包含三个 Python 函数。我们在 CPU、DPU 和跨 PU 三种情形下比较基线 (Molecule-homo)、Molecule 和 HeteroK8s-FaaS。跨 PU 情形保证所有函数间调用都跨越 PU; 例如 Alexa 的第 1、3、5 个函数位于主机 CPU, 第 2、4 个函数位于 DPU。全部实例均为热启动, 不计冷启动成本。结果见图 10。与基线相比, Molecule 在 Alexa 上将端到端延迟降低 $1.94\text{--}2.32\times$, 在 MapReduce 上降低 $4.20\text{--}4.21\times$; HeteroK8s-FaaS 的延迟进一步改善, 在 Alexa 上降低 $1.96\text{--}2.34\times$, 在 MapReduce 上降低 $4.19\text{--}4.45\times$ 。

5.3 案例研究: 卸载调度器

动机。 启动延迟是云原生应用最重要的指标之一 [88]。虽然已有许多工作使用检查点/恢复、fork 和缓存来优化容器启动延迟, 但我们观察到镜像拉取仍带来显著挑战: 如果请求被调度到尚未准备相应镜像的节点, 平台必须先拉取镜像, 再启动实例。为分析这项成本, 我们使用 Kperf 在广泛采用的 FaaS 系统 Knative[29] 上评估响应延迟。如图 11(a) 所示, 镜像未缓存时的延迟约为已缓存时的 $10\times$ 。

思路与挑战。 云原生平台通常依赖全局调度器为请求选择节点, 例如 Kubernetes 的 kube-scheduler。一种思路是采用镜像感知调度器, 优先把请求分配给已经准备好镜像的节点; Kubernetes 就支持镜像局部性策略 [30]。然而, 这要求调度器维护全部节点的镜像信息。在真实环境中, 缓存所有镜像信息需要 1GB–10GB 资源, 而且镜像状态更新会通知调度器, 使调度器成为控制面瓶颈。

解决方案。 HeteroK8s 可以把调度器中的镜像缓存卸载到 DPU, 利用 DPU 的空余资源维护镜像信息。此外, DPU 更接近网络, 由它更新镜像状态也更合理。在此卸载基础上, 我们设计了镜像与模板感知策略 (容器 fork 的模板见文献 [44]), 并将其实现为 kube-scheduler 插件, 优先选择已经具有模板实例和缓存镜像的节点。HeteroK8s 通过卸载后的调度器与新策略支持未经修改的 Knative。如图 11(b) 所示, 我们分析由四台机器组成的完整集群的平均延迟; HeteroK8s-Kn 将平均延迟从 481ms 降至 33ms,

表 1: 受支持应用的代码行数。HeteroNet 能支持代码规模显著更大的真实应用。✓表示 Molecule 或 HeteroNet 支持, ✗表示不支持; CNN image 来自 FunctionBench, Alexa 来自 ServerlessBench。

应用	版本	LoC	Molecule	HeteroNet
Python3	3.8.10	1,085,262	✗	✓
Envoy	#330dbdc8f9	764,457	✗	✓
CNN image	#296f5f2	135	✓	✓
Alexa smarhome	#207b345	1,804	✓	✓

即降低 14.4×。

6 HeteroNet 评估

6.1 hetero-socket 性能

我们评估 hetero-socket 的网络性能。测试使用 LMBench[67] 中的 `lat_tcp` 和 `bw_tcp`, 分别测量网络延迟和吞吐量, 并比较三种系统: (1) Baseline, 即单个 PU 上使用 Linux 网络时的性能; (2) Reserve, 即为每条连接预留特定大小 GShm 的 HeteroNet 版本, 例如 1MB 表示连接为两个方向各预留一个 1MB 缓冲区; (3) HeteroNet, 为本地记录使用 128KB, 为共享区域使用 128MB。

延迟。 图 12(a)(b) 给出基线与 HeteroNet 的延迟, 其中本地通信使用共享内存, XPU 通信使用 CXL。对于同一 PU 内的小消息, HeteroNet 相较基线将延迟降低 $10.1\times$ (2KB) 到 $31.9\times$ (16B); 对于大消息, 降低 $2.1\times$ (256KB) 到 $13.2\times$ (8KB)。对于跨 PU 情形, 基于 CXL 的 HeteroNet 可达到与共享内存相当的延迟, 并比跨 PU 网络基线低两个数量级 (因篇幅未在图中展示)。使用 CXL 时, U-NAPI 的一次跨 PU 通知约耗时 $0.51\mu s$, 此外还需处理异常与上调的开销。

吞吐量。 图 12(c)(d) 给出 PU 内吞吐量, 其中 HeteroNet 使用共享内存; 图 12(e)(f) 给出跨 PU 吞吐量, 其中 HeteroNet 使用 CXL。由于跨 PU 网络慢得多, 基线仍使用 PU 内结果。图中还给出采用固定预留缓冲区、而不是本地记录与共享区域设计的 HeteroNet 结果。与基线相比, 单 PU 上 HeteroNet 对小消息的吞吐量提升 $1.9\sim 2.9\times$, 对大消息提升 $4.8\sim 6.3\times$; 跨 PU 的 CXL 版 HeteroNet 也比单 PU 基线高 $1.4\sim 4.4\times$ 。

资源成本。 假设图 12(c)–(f) 中存在 1,000 条连接。每条连接预留 16MB 将占用 16GB 固定通信资源, 是不可忽视的负担。每条连接只用 1MB (总计 1GB) 或 256KB (1,000 条连接共享总计 256MB) 时, 小消息仍可达到良好吞吐量, 但消息尺寸变大后会显著变慢, 例如 256KB 配置慢超过 $2\times$ 。HeteroNet 使用 128KB 私有记录和 128MB 共享区域 (总计 256MB), 性能却几乎与预留 16MB 相同, 在该情形中节省 $64\times$ 内存。我们在真实云环境中的实践是为 RDMA 每条连接预留 1MB, 并为传统 Linux TCP 再预留 1MB, 因此这一改善十分显著。

6.2 兼容性

表 1 比较 HeteroK8s 与 Molecule[44] 所支持的两类最复杂应用的代码行数。Molecule 等框架虽然通过新抽象成功支持无服务器计算, 却很难支持基于 Python 的服务和 Envoy[22] 等通用云原生应用; Envoy 在服务网格中被

广泛采用。HeteroK8s 借助 HeteroNet, 能够在 CPU–DPU 计算机上支持代码行数显著更高的通用应用, 形成一个新的里程碑。

支持其他异构设备。 HeteroNet 可以推广到其他通用异构设备。但对于 FPGA、GPU 等领域专用加速器, 应用通常不依赖套接字之类的 API; 这时可以直接使用 DMA 等低级抽象进行高效通信。未来也可以结合 GPUUnet[56]、Lynx[83] 和 Coyote[57] 等相关系统扩展 HeteroNet。

支持其他系统。 我们基于 Kubernetes 实现原型, 因为它是目前使用最广泛的云原生平台 [20]。不过, 动态拆分的思想以及 `hetero-netns`、`hetero-socket` 两项技术具有通用性; 实现也可以用于云原生应用之外的其他场景。

局限。 当前原型支持通过 `libc` 动态链接的应用使用 `hetero-socket`, 但不支持预先静态编译的应用; 通过重新编译可以支持后者。HeteroNet 主要关注通过网络协作的应用。既有工作 [44] 已经支持 IPC 情形, 未来将把该能力集成进来。线程等更细粒度的卸载仍是有待未来解决的开放问题。XPU 的更多策略和收益, 例如能效 [65], 也值得探索, 但它们与本文所提供的动态拆分机制相互正交。

7 相关工作

异构计算机的系统支持。 已有大量相关研究 [33, 35–37, 40, 41, 47, 49, 56, 64, 72, 74, 76, 78–81, 83]。分布式操作系统 [36, 37, 80] 使用包含多个内核的单一操作系统来管理异构设备。另一些系统使用智能设备卸载特定任务 [49, 56, 76, 78, 81]。Floem[76] 为 NIC 加速应用提出编程抽象; LeapIO[62] 使用协处理器卸载云存储服务; E3[65] 是面向 SmartNIC 加速服务器的微服务执行平台, 可显著提高能效。作者认为, HeteroNet 是首个利用网络抽象实现动态拆分的系统; HeteroK8s 上的实践证明了该方法的可行性与效率。

优化跨 PU 通信。 既有工作 [60, 63, 65, 71] 已经探索利用 PCIe/DMA 高效实现 SmartNIC 与主机通信。GPUUnet[56] 为 GPU 程序提出套接字抽象; Solros[69] 和 SPIN[39] 使用 P2P DMA 优化通信; Pond[61] 利用 CXL 实现内存池化; CXL-Shm[90] 利用 CXL 实现高效分布式共享内存, 并以引用计数处理部分故障。作者认为, HeteroNet 是其中首个提供网络抽象的系统。

相关网络方案。 SocksDirect[59] 提出用户空间套接字系统, 使用用户态监控器管理资源与事件; 但它并非针对 XPU 计算机, 仍依赖预留内存缓冲区。SMC-R[46] 在内核中支持基于 RDMA 共享内存的网络, 能够有效管理资源并支持商用应用, 但以内核为中心的设计会产生用户态与内核态之间切换、复制的成本。IX[38] 提出数据面操作系统, 同时提供高 I/O 性能和保护。Arrakis[75]、mTCP[52]、F-stack[23] 等同样保留 POSIX API 的内核旁路网络库, 通常采用高效的非用户空间 TCP/IP 实现和 DPDK。MegaPipe[50] 与 StackMap[87] 则使用零复制 API 优化网络栈。HeteroNet 延续这一研究方向, 并提出推测式分配与 U-NAPI, 在异构计算机中平衡性能和资源效率。

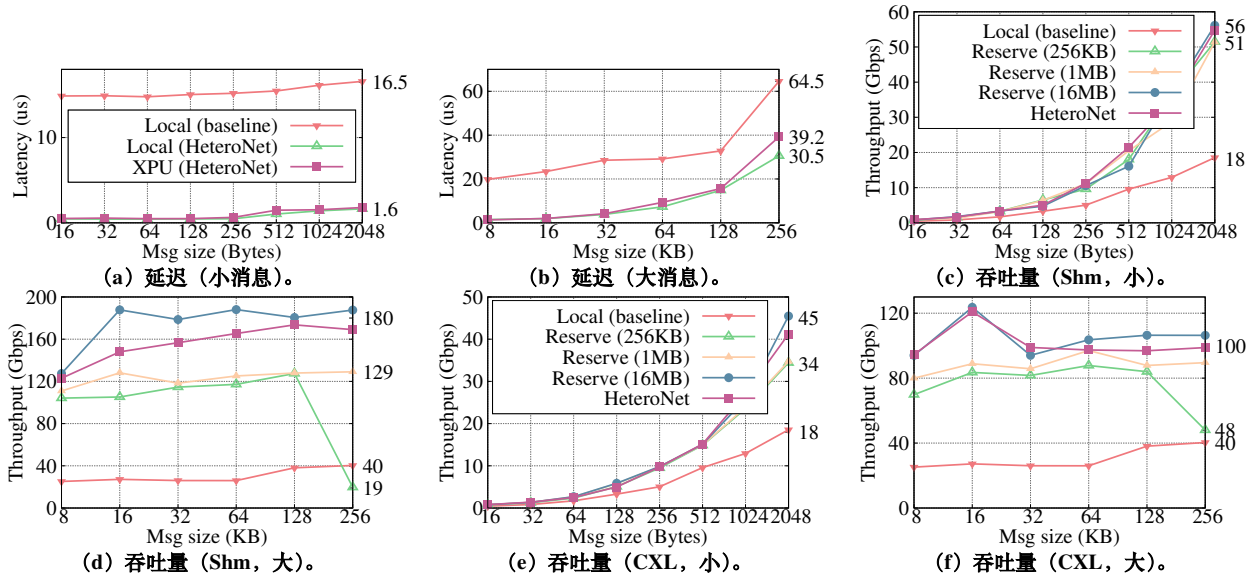


图 12: 网络性能。

8 结论

本文提出由 HeteroNet 支撑的 HeteroPod，使云原生应用能够在 CPU-DPU 计算机上进行动态拆分。HeteroNet 包含用于提供统一网络命名空间的 `hetero-netns`，以及同时取得内核旁路性能与内核辅助资源效率的 `hetero-socket`。我们构建 HeteroK8s，并在无服务器计算和微服务等代表性云原生场景中展示其收益。作者计划开源 HeteroNet。

参考文献

- [1] 2017. Service mesh: A critical component of the cloud native stack. <https://www.cncf.io/blog/2017/04/26/service-mesh-critical-component-cloud-native-stack/>. Referenced April 2023.
- [2] 2021. Amazon EC2 F1 Instances. <https://aws.amazon.com/ec2/instance-types/f1>. Referenced July 2021.
- [3] 2021. AWS Lambda - Serverless Compute. <https://aws.amazon.com/lambda/>. Referenced Jan. 2021.
- [4] 2021. Azure Functions Serverless Architecture. <https://azure.microsoft.com/en-us/services/functions/>. Referenced Jan. 2021.
- [5] 2021. Cloud Functions - Overview | IBM. <https://www.ibm.com/cloud/functions>. Referenced Jan. 2021.
- [6] 2021. Compute Express Link. <https://www.computeexpresslink.org/>. Referenced Aug. 2021.
- [7] 2021. DPDK. <https://www.dpdk.org/>.
- [8] 2021. FPGA Accelerated Cloud Server-HUAWEI CLOUD. <https://www.huaweicloud.com/en-us/product/fcs.html>. Referenced Nov. 2021.
- [9] 2021. FPGA Cloud Compute. <https://cloud.baidu.com/product/fpga.html>. Referenced Aug. 2021.
- [10] 2021. Google Cloud Function. <https://cloud.google.com/functions/>. Referenced Jan. 2021.
- [11] 2021. Hardware Acceleration over NFV in China Mobile. https://wiki.opnfv.org/download/attachments/20745096/opnfv_Acc.pdf. Referenced Nov. 2021.
- [12] 2021. Instance families - Instance | Alibaba Cloud Documentation Center. <https://www.alibabacloud.com/help/doc-detail/25378.htm>. Referenced Aug. 2021.
- [13] 2021. Linux manual page of LD_PRELOAD. <https://man7.org/linux/man-pages/man8/ld.so.8.html>. man page of LD_PRELOAD.
- [14] 2021. Marvell OCTEON SDK. <https://www.marvell.com/content/dam/marvell/en/public-collateral/embedded-processors/marvell-octeon-tx2-sdk-solutions-brief.pdf>. Referenced April 2021.
- [15] 2021. Multi-Core Processors - LiquidIO Smart NICs | Network adapter - Marvell. <https://www.marvell.com/products/infrastructure-processors/multi-core-processors/liquidio-smart-nics.html>. Referenced 2021.
- [16] 2021. Xilinx Powers Huawei FPGA Accelerated Cloud Server. <https://www.xilinx.com/news/press/2017/xilinx-powers-huawei-fpga-accelerated-cloud-server.html>. Referenced Aug. 2021.
- [17] 2022. Introducing Ambient Mesh. <https://istio.io/latest/blog/2022/introducing-ambient-mesh/>. Referenced Mar 2023.
- [18] 2023. Accelerating Cloud-Ready Infrastructure and Kubernetes with Red Hat OpenShift and the NVIDIA BlueField DPU. <https://developer.nvidia.com/blog/accelerating-cloud-ready-infrastructure-and-kubernetes-with-red-hat-openshift-and-bluefield-dpu/>. Referenced April 2023.
- [19] 2023. Cloud Native Computing Foundation. <https://www.cncf.io/about/>. Referenced Mar 2023.
- [20] 2023. CNCF Cloud Native Interactive Landscape. <https://landscape.cncf.io/>. Referenced Nov 2023.

- [21] 2023. Emoji.voto. <https://github.com/BuoyantIO/emojivoto>. Referenced Mar 2023.
- [22] 2023. Envoy Proxy. <https://www.envoyproxy.io>. Referenced 2023.
- [23] 2023. F-Stack: High Performance Network Framework Based On DPDK. <http://www.f-stack.org/>. Referenced Apr. 2023.
- [24] 2023. Istio: Simplify observability, traffic management, security, and policy with the leading service mesh. <https://istio.io/>. Referenced Mar 2023.
- [25] 2023. Linux manual page of network_namespaces. https://man7.org/linux/man-pages/man7/network_namespaces.7.html. man page of network_namespaces.
- [26] 2023. Multi-platform images. <https://docs.docker.com/build/building/multi-platform/>. Referenced Nov 2023.
- [27] 2023. NVIDIA BlueField DPU Container Deployment Guide. <https://docs.nvidia.com/doca/archive/doca-v1.2/pdf/container-deployment.pdf>. Referenced April 2023.
- [28] 2023. rsocket(7): RDMA socket API - Linux man page. <https://linux.die.net/man/7/rsocket>. Referenced Mar 2023.
- [29] 2024. Knative. <https://knative.dev/docs/>.
- [30] 2024. Kubernetes image locality policy. <https://pkg.go.dev/k8s.io/kubernetes/pkg/scheduler/framework/plugins/imagelocality>.
- [31] 2024. Mellanox libvma. <https://github.com/mellanox/libvma>. Referenced April 2024.
- [32] 2025. NAPI Document. <https://docs.kernel.org/networking/napi.html>.
- [33] Reto Achermann, David Cock, Roni Haecki, Nora Hossle, Lukas Humbel, Timothy Roscoe, and Daniel Schwyn. 2021. Mmapx: Uniform Memory Protection in a Heterogeneous World. In *Proceedings of the Workshop on Hot Topics in Operating Systems* (Ann Arbor, Michigan) (*HotOS '21*). Association for Computing Machinery, New York, NY, USA, 159–166. doi:10.1145/3458336.3465273
- [34] Istemi Ekin Akkus, Ruichuan Chen, Ivica Rimac, Manuel Stein, Klaus Satzke, Andre Beck, Paarijaat Aditya, and Volker Hilt. 2018. *SAND: Towards High-Performance Serverless Computing*. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. 923–935.
- [35] Nils Asmussen, Marcus Völz, Benedikt Nöthen, Hermann Härtig, and Gerhard Fettweis. 2016. M3: A Hardware/Operating-System Co-Design to Tame Heterogeneous Manycores. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems* (Atlanta, Georgia, USA) (*ASPLOS '16*). ACM, New York, NY, USA, 189–203. doi:10.1145/2872362.2872371
- [36] Antonio Barbalace, Marina Sadini, Saif Ansary, Christopher Jeleznianski, Akshay Ravichandran, Cagil Kendir, Alastair Murray, and Binoy Ravindran. 2015. Popcorn: Bridging the Programmability Gap in Heterogeneous-ISA Platforms. In *Proceedings of the Tenth European Conference on Computer Systems* (Bordeaux, France) (*EuroSys '15*). Association for Computing Machinery, New York, NY, USA, Article 29, 16 pages. doi:10.1145/2741948.2741962
- [37] Andrew Baumann, Paul Barham, Pierre-Evariste Dagand, Tim Harris, Rebecca Isaacs, Simon Peter, Timothy Roscoe, Adrian Schüpbach, and Akhilesh Singhanian. 2009. The Multikernel: A New OS Architecture for Scalable Multicore Systems. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (Big Sky, Montana, USA) (*SOSP '09*). Association for Computing Machinery, New York, NY, USA, 29–44. doi:10.1145/1629575.1629579
- [38] Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. 2014. IX: A protected dataplane operating system for high throughput and low latency. In *Proceedings of the 11th USENIX Symposium on Operating System Design and Implementation (OSDI)*. USENIX.
- [39] Shai Bergman, Tanya Brokhman, Tzachi Cohen, and Mark Silberstein. 2017. *SPIN: Seamless Operating System Integration of Peer-to-Peer {DMA} Between SSDs and GPUs*. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. 167–179.
- [40] Tanya Brokhman, Pavel Lifshits, and Mark Silberstein. 2019. *GAIA: An OS Page Cache for Heterogeneous Systems*. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. 661–674.
- [41] Shengsun Cho, Han Chen, Sergey Madaminov, Michael Ferdman, and Peter Milder. 2020. Flick: Fast and Lightweight ISA-Crossing Call for Heterogeneous-ISA Environments. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture* (Virtual Event) (*ISCA '20*). IEEE Press, 187–198. doi:10.1109/ISCA45697.2020.00026
- [42] Marcin Copik, Konstantin Taranov, Alexandru Calotoiu, and Torsten Hoefer. 2022. rFaaS: Enabling High Performance Serverless with RDMA and Decentralization. arXiv:2106.13859 [cs.DC]
- [43] Lazar Cvetković, François Costa, Mihajlo Djokic, Michal Friedman, and Ana Klimovic. 2024. Dirigent: Lightweight Serverless Orchestration. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (*SOSP '24*). Association for Computing Machinery, New York, NY, USA, 369–384. doi:10.1145/3694715.3695966
- [44] Dong Du, Qingyuan Liu, Xueqiang Jiang, Yubin Xia, Binyu Zang, and Haibo Chen. 2022. Serverless Computing on Heterogeneous Computers. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (*ASPLOS '22*). Association for Computing Machinery, New York, NY, USA, 797–813. doi:10.1145/3503222.3507732
- [45] Daniel Firestone, Andrew Putnam, Sambrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat,

- Vivek Bhanu, Adrian M. Caulfield, Eric S. Chung, Harish Kumar Chandrappa, Somesh Chaturmohta, Matt Humphrey, Jack Lavier, Norman Lam, Fengfen Liu, Kalin Ovtcharov, Jitu Padhye, Gautham Popuri, Shachar Raindel, Tejas Sapre, Mark Shaw, Gabriel Silva, Madhan Sivakumar, Nisheeth Srivastava, Anshuman Verma, Qasim Zuhair, Deepak Bansal, Doug Burger, Kushagra Vaid, David A. Maltz, and Albert G. Greenberg. 2018. Azure Accelerated Networking: SmartNICs in the Public Cloud. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. 51–66.
- [46] Mike Fox, Constantinos (Gus) Kassimis, and Jerry Stevens. 2015. IBM’s Shared Memory Communications over RDMA (SMC-R) Protocol. RFC 7609. [doi:10.17487/RFC7609](https://doi.org/10.17487/RFC7609)
- [47] Benjamin Gamsa, Orran Krieger, Jonathan Appavoo, and Michael Stumm. 1999. Tornado: Maximizing locality and concurrency in a shared memory multiprocessor operating system. In *OSDI*, Vol. 99. 87–100.
- [48] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (*ASPLOS ’19*). Association for Computing Machinery, New York, NY, USA, 3–18. [doi:10.1145/3297858.3304013](https://doi.org/10.1145/3297858.3304013)
- [49] Boncheol Gu, Andre S. Yoon, Duck-Ho Bae, Insoon Jo, Jinyoung Lee, Jonghyun Yoon, Jeong-Uk Kang, Moonsang Kwon, Chanho Yoon, Sangyeun Cho, Jaeheon Jeong, and Duckhyun Chang. 2016. Biscuit: A Framework for near-Data Processing of Big Data Workloads. In *Proceedings of the 43rd International Symposium on Computer Architecture* (Seoul, Republic of Korea) (*ISCA ’16*). IEEE Press, 153–165. [doi:10.1109/ISCA.2016.23](https://doi.org/10.1109/ISCA.2016.23)
- [50] Sangjin Han, Scott Marshall, Byung-Gon Chun, and Sylvia Ratnasamy. 2012. MegaPipe: a new programming interface for scalable network I/O. In *Proceedings of the 10th USENIX Conference on Operating Systems Design and Implementation* (Hollywood, CA, USA) (*OSDI ’12*). USENIX Association, USA, 135–148.
- [51] Jialiang Huang, MingXing Zhang, Teng Ma, Zheng Liu, Sixing Lin, Kang Chen, Jinlei Jiang, Xia Liao, Yingdi Shan, Ning Zhang, Mengting Lu, Tao Ma, Haifeng Gong, and Yongwei Wu. 2024. TrEnv: Transparently Share Serverless Execution Environments Across Different Functions and Nodes. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (*SOSP ’24*). Association for Computing Machinery, New York, NY, USA, 421–437. [doi:10.1145/3694715.3695967](https://doi.org/10.1145/3694715.3695967)
- [52] Eun Young Jeong, Shinae Woo, Muhammad Jamshed, Hae-won Jeong, Sunghwan Ihm, Dongsu Han, and Kyoungsoo Park. 2014. MTCP: A Highly Scalable User-Level TCP Stack for Multicore Systems. In *Proceedings of the 11th USENIX Conference on Networked Systems Design and Implementation* (Seattle, WA) (*NSDI ’14*). USENIX Association, USA, 489–502.
- [53] Zhipeng Jia and Emmett Witchel. 2021. Nightcore: Efficient and Scalable Serverless Computing for Latency-Sensitive, Interactive Microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) (*ASPLOS 2021*). Association for Computing Machinery, New York, NY, USA, 152–166. [doi:10.1145/3445814.3446701](https://doi.org/10.1145/3445814.3446701)
- [54] Jongyul Kim, Insu Jang, Waleed Reda, Jaeseong Im, Marco Canini, Dejan Kostić, Youngjin Kwon, Simon Peter, and Emmett Witchel. 2021. LineFS: Efficient SmartNIC Offload of a Distributed File System with Pipeline Parallelism. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (Virtual Event, Germany) (*SOSP ’21*). Association for Computing Machinery, New York, NY, USA, 756–771. [doi:10.1145/3477132.3483565](https://doi.org/10.1145/3477132.3483565)
- [55] Jeongchul Kim and Kyungyong Lee. 2019. Practical Cloud Workloads for Serverless FaaS. In *Proceedings of the ACM Symposium on Cloud Computing* (Santa Cruz, CA, USA) (*SoCC ’19*). Association for Computing Machinery, New York, NY, USA, 477. [doi:10.1145/3357223.3365439](https://doi.org/10.1145/3357223.3365439)
- [56] Sangman Kim, Seonggu Huh, Xinya Zhang, Yige Hu, Amir Wated, Emmett Witchel, and Mark Silberstein. 2014. GPUnet: Networking Abstractions for GPU Programs. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. 201–216.
- [57] Dario Korolija, Timothy Roscoe, and Gustavo Alonso. 2020. Do OS abstractions make sense on FPGAs?. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 991–1010.
- [58] Swaroop Kotni, Ajay Nayak, Vinod Ganapathy, and Arkaprava Basu. 2021. Faastlane: Accelerating Function-as-a-Service Workflows. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 805–820. <https://www.usenix.org/conference/atc21/presentation/kotni>
- [59] Bojie Li, Tianyi Cui, Zibo Wang, Wei Bai, and Lintao Zhang. 2019. Socksdirect: datacenter sockets can be fast and compatible. In *Proceedings of the ACM Special Interest Group on Data Communication* (Beijing, China) (*SIGCOMM ’19*). Association for Computing Machinery, New York, NY, USA, 90–103. [doi:10.1145/3341302.3342071](https://doi.org/10.1145/3341302.3342071)
- [60] Bojie Li, Kun Tan, Layong (Larry) Luo, Yanqing Peng, Renqian Luo, Ningyi Xu, Yongqiang Xiong, Peng Cheng, and Enhong Chen. 2016. ClickNP: Highly Flexible and High Performance Network Processing with Reconfigurable Hardware. In *Proceedings of the 2016 ACM SIGCOMM Conference* (Florianopolis, Brazil) (*SIGCOMM ’16*). Association for Computing Machinery, New York, NY, USA, 1–14. [doi:10.1145/2934872.2934897](https://doi.org/10.1145/2934872.2934897)
- [61] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. 2023. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) (*ASPLOS 2023*). Association for Computing Machinery, New York, NY, USA, 574–587. [doi:10.1145/3575693.3578835](https://doi.org/10.1145/3575693.3578835)

- [62] Huaicheng Li, Mingzhe Hao, Stanko Novakovic, Vaibhav Gogte, Sriram Govindan, Dan R. K. Ports, Irene Zhang, Riccardo Bianchini, Haryadi S. Gunawi, and Anirudh Badam. 2020. LeapIO: Efficient and Portable Virtual NVMe Storage on ARM SoCs. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, New York, NY, USA, 591–605. <https://doi.org/10.1145/3373376.3378531>
- [63] Felix Xiaozhu Lin and Xu Liu. 2016. Memif: Towards Programming Heterogeneous Memory Asynchronously. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems* (Atlanta, Georgia, USA) (ASPLOS '16). Association for Computing Machinery, New York, NY, USA, 369–383. [doi:10.1145/2872362.2872401](https://doi.org/10.1145/2872362.2872401)
- [64] Ming Liu, Tianyi Cui, Henry Schuh, Arvind Krishnamurthy, Simon Peter, and Karan Gupta. 2019. Offloading distributed applications onto smartNICs using iPipe. In *Proceedings of the ACM Special Interest Group on Data Communication*. 318–333.
- [65] Ming Liu, Simon Peter, Arvind Krishnamurthy, and Phitchaya Mangpo Phothilimthana. 2019. E3: Energy-Efficient Microservices on SmartNIC-Accelerated Servers. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, Renton, WA, 363–378. <https://www.usenix.org/conference/atc19/presentation/liu-ming>
- [66] Qingyuan Liu, Dong Du, Yubin Xia, Ping Zhang, and Haibo Chen. 2023. The Gap Between Serverless Research and Real-world Systems. In *Proceedings of the 2023 ACM Symposium on Cloud Computing* (Santa Cruz, CA, USA) (SoCC '23). Association for Computing Machinery, New York, NY, USA, 475–485. [doi:10.1145/3620678.3624785](https://doi.org/10.1145/3620678.3624785)
- [67] Larry W McVoy, Carl Staelin, et al. 1996. Imbench: Portable Tools for Performance Analysis.. In *USENIX annual technical conference*. San Diego, CA, USA, 279–294.
- [68] Mellanox Technologies. 2021. BlueField Multicore System on Chip. http://www.mellanox.com/related-docs/npu-multicore-processors/PB_BlueField_Storage_Controller_Card.pdf.
- [69] Changwoo Min, Woonhak Kang, Mohan Kumar, Sanidhya Kashyap, Steffen Maass, Heeseung Jo, and Taesoo Kim. 2018. Solros: a data-centric operating system architecture for heterogeneous computing. In *Proceedings of the Thirteenth EuroSys Conference*. ACM, 36.
- [70] Netronome. 2021. Netronome Agilio SmartNICs. https://www.netronome.com/media/documents/PB_NFP-4000.pdf.
- [71] Rolf Neugebauer, Gianni Antichi, José Fernando Zazo, Yury Audzevich, Sergio López-Buedo, and Andrew W. Moore. 2018. Understanding PCIe Performance for End Host Networking. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication* (Budapest, Hungary) (SIGCOMM '18). Association for Computing Machinery, New York, NY, USA, 327–341. [doi:10.1145/3230543.3230560](https://doi.org/10.1145/3230543.3230560)
- [72] Joel Nider and Alexandra (Sasha) Fedorova. 2021. The Last CPU. In *Proceedings of the Workshop on Hot Topics in Operating Systems* (Ann Arbor, Michigan) (HotOS '21). Association for Computing Machinery, New York, NY, USA, 1–8. [doi:10.1145/3458336.3465291](https://doi.org/10.1145/3458336.3465291)
- [73] NVIDIA Mellanox. 2023. NVIDIA BlueField-3 DPU User Guide. <https://docs.nvidia.com/networking/display/BlueField3DPU/Introduction>.
- [74] Nathan Pemberton, Johann Schleier-Smith, and Joseph E. Gonzalez. 2021. The RESTless Cloud. In *Proceedings of the Workshop on Hot Topics in Operating Systems* (Ann Arbor, Michigan) (HotOS '21). Association for Computing Machinery, New York, NY, USA, 49–57. [doi:10.1145/3458336.3465280](https://doi.org/10.1145/3458336.3465280)
- [75] Simon Peter, Jialin Li, Irene Zhang, Dan RK Ports, Doug Woos, Arvind Krishnamurthy, Thomas Anderson, and Timothy Roscoe. 2015. Arrakis: The operating system is the control plane. *ACM Transactions on Computer Systems (TOCS)* 33, 4 (2015), 1–30.
- [76] Phitchaya Mangpo Phothilimthana, Ming Liu, Antoine Kaufmann, Simon Peter, Rastislav Bodik, and Thomas Anderson. 2018. Floem: a programming system for NIC-accelerated network applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 663–679.
- [77] Francisco Romero, Mark Zhao, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2021. Llama: A Heterogeneous & Serverless Framework for Auto-Tuning Video Analytics Pipelines. In *Proceedings of the ACM Symposium on Cloud Computing* (Seattle, WA, USA) (SoCC '21). Association for Computing Machinery, New York, NY, USA, 1–17. [doi:10.1145/3472883.3486972](https://doi.org/10.1145/3472883.3486972)
- [78] Sudharsan Seshadri, Mark Gahagan, Sundaram Bhaskaran, Trevor Bunker, Arup De, Yanqin Jin, Yang Liu, and Steven Swanson. 2014. Willow: A User-Programmable SSD. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. 67–80.
- [79] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiying Zhang. 2018. LegoOS: A Disseminated, Distributed OS for Hardware Resource Disaggregation. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 69–87.
- [80] Mark Silberstein. 2017. OmniX: An Accelerator-Centric OS for Omni-Programmable Systems. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems* (Whistler, BC, Canada) (HotOS '17). Association for Computing Machinery, New York, NY, USA, 69–75. [doi:10.1145/3102980.3102992](https://doi.org/10.1145/3102980.3102992)
- [81] Mark Silberstein, Bryan Ford, Idit Keidar, and Emmett Witchel. 2013. GPUfs: Integrating a File System with GPUs. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems* (Houston, Texas, USA) (ASPLOS '13). Association for Computing Machinery, New York, NY, USA, 485–498. [doi:10.1145/2451116.2451169](https://doi.org/10.1145/2451116.2451169)
- [82] Ariel Szekely, Adam Belay, Robert Morris, and M. Frans Kaashoek. 2024. Unifying serverless and microservice workloads with SigmaOS. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX,

USA) (*SOSP '24*). Association for Computing Machinery, New York, NY, USA, 385–402. doi:10.1145/3694715.3695947

- [83] Maroun Tork, Lina Maudlej, and Mark Silberstein. 2020. *Lynx: A SmartNIC-Driven Accelerator-Centric Architecture for Network Servers*. Association for Computing Machinery, New York, NY, USA, 117–131. <https://doi.org/10.1145/3373376.3378528>
- [84] Xingda Wei, Rongxin Chen, Yuhan Yang, Rong Chen, and Haibo Chen. 2022. A Comprehensive Study on Off-path SmartNIC. arXiv:2212.07868 [cs.DC]
- [85] Xingda Wei, Fangming Lu, Rong Chen, and Haibo Chen. 2022. KRCORE: A Microsecond-scale RDMA Control Plane for Elastic Computing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA, 121–136. <https://www.usenix.org/conference/atc22/presentation/wei>
- [86] Xingda Wei, Fangming Lu, Tianxia Wang, Jinyu Gu, Yuhan Yang, Rong Chen, and Haibo Chen. 2022. No Provisioned Concurrency: Fast RDMA-coded Remote Fork for Serverless Computing. arXiv:2203.10225 [cs.OS]
- [87] Kenichi Yasukata, Michio Honda, Douglas Santry, and Lars Eggert. 2016. StackMap: Low-Latency Networking with the OS Stack and Dedicated NICs. In *2016 USENIX Annual Technical Conference (USENIX ATC 16)*. USENIX Association, Denver, CO, 43–56. <https://www.usenix.org/conference/atc16/technical-sessions/presentation/yasukata>
- [88] Tianyi Yu, Qingyuan Liu, Dong Du, Yubin Xia, Binyu Zang, Ziqian Lu, Pingchao Yang, Chenggang Qin, and Haibo Chen. 2020. Characterizing Serverless Platforms with Serverless-Bench. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '20)*. Association for Computing Machinery. doi:10.1145/3419111.3421280
- [89] Irene Zhang, Amanda Raybuck, Pratyush Patel, Kirk Olynyk, Jacob Nelson, Omar S. Navarro Leija, Ashlie Martinez, Jing Liu, Anna Kornfeld Simpson, Sujay Jayakar, Pedro Henrique Penna, Max Demoulin, Piali Choudhury, and Anirudh Badam. 2021. The Demikernel Datapath OS Architecture for Microsecond-Scale Datacenter Systems. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (Virtual Event, Germany) (SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 195–211. doi:10.1145/3477132.3483569
- [90] Mingxing Zhang, Teng Ma, Jinqi Hua, Zheng Liu, Kang Chen, Ning Ding, Fan Du, Jinlei Jiang, Tao Ma, and Yongwei Wu. 2023. Partial Failure Resilient Memory Management System for (CXL-based) Distributed Shared Memory. In *Proceedings of the 29th Symposium on Operating Systems Principles (Koblenz, Germany) (SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 658–674. doi:10.1145/3600006.3613135